

Ejercicio 3

1. Determine cómo evoluciona la medida CPI cuando incrementa el valor de X, analice el resultado.

Para poder determinar la evolución de la medida de CPI al momento de incrementar el valor de X, se utilizó el código de la carpeta “implementacion_pseudo_code” disponible en este mismo repositorio, modificando el valor de X manualmente al momento de ejecutar el código.

Cabe destacar que para poder determinar un valor de mayor precisión, se ejecutó un ciclo *for* vacío para calcular cuántos ciclos tenía este. Una vez teniendo aquel valor, se pudo ejecutar todas las operaciones y calcular sus ciclos respectivos y luego restar el valor guardado de *overhead* del ciclo *for* vacío. Además, se realizó el mismo proceso de cambiar X en las diversas frecuencias incluidas en la ESP32-S3 (80, 160 y 240 MHz) para determinar si el comportamiento era similar.

A continuación se presentan los gráficos con los datos obtenidos:

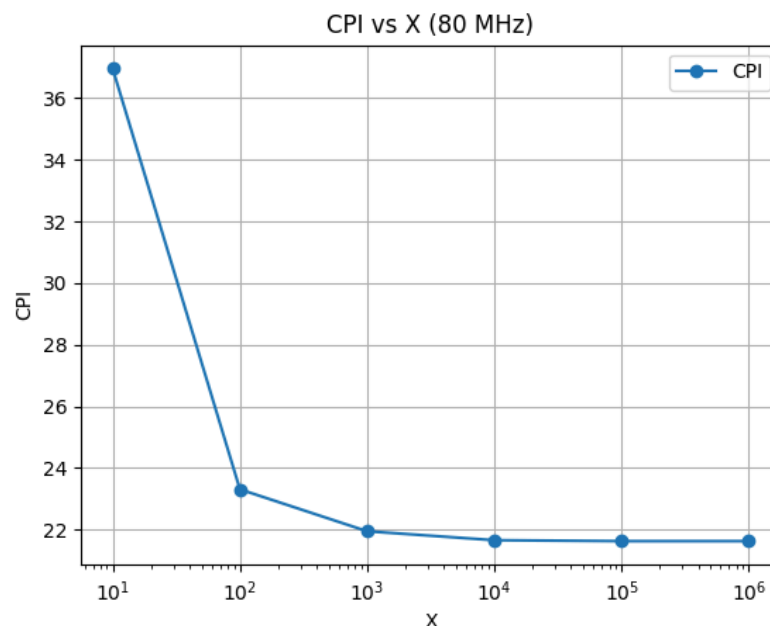


Figura 1: Gráfico de CPI según valor de X (80 MHz)

Fuente: Elaboración propia

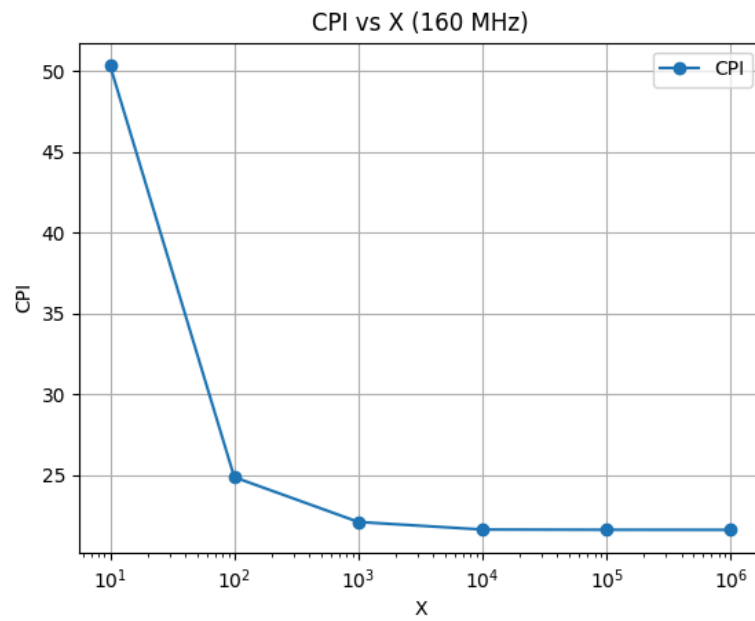


Figura 2: Gráfico de CPI según valor de X (160 MHz)
Fuente: Elaboración propia

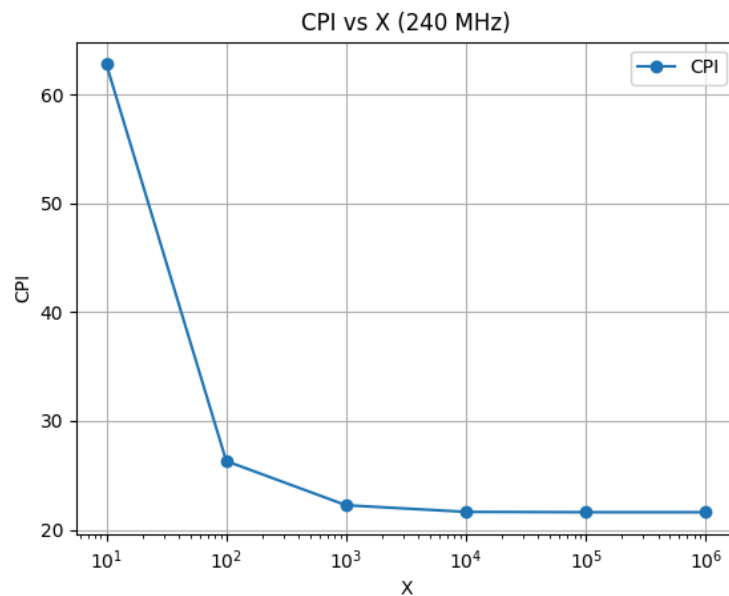


Figura 3: Gráfico de CPI según valor de X (240 MHz)
Fuente: Elaboración propia

Como se puede observar en los gráficos de las figuras 1, 2 y 3, a pesar de modificar la frecuencia, la tendencia y comportamiento de la medida de CPI

en base a la cantidad de operaciones es la misma. Se puede apreciar que, en un principio a un muy bajo nivel de operaciones ($X = 10$), el CPI mantiene un valor muy elevado, para luego al ir aumentando el valor de X (hasta 1 millón), es evidente como el valor baja drásticamente hasta estabilizarse.

Los valores iniciales de CPI varían según la frecuencia determinada en *menuconfig*, y se puede determinar que, a mayor frecuencia, mayor el valor de CPI inicial. Considerando $X = 10$, 80 MHz tuvo un valor de CPI de 36.96, 160 MHz de 50.34 y 240 MHz de 62.78 pero esto no está necesariamente relacionado a la frecuencia si no al bajo valor de X .

Cuando X toma un valor más pequeño, hay una menor cantidad de instrucciones y los costos estándar de ejecución como *overhead*, latencias o no tener información cacheada, toman mayor peso y se dispara el valor. Debido a que CPI se calcula dividiendo Ciclos/Instrucción, teniendo una menor cantidad de instrucciones, el denominador es pequeño y permite que aumente el valor considerablemente.

A medida que X aumenta, el número de instrucciones ejecutadas también lo hace y disminuye o contrarresta el efecto de los valores de latencia o ciclos estándar del sistema, llevándolo a disminuir bastante hasta estabilizarse.

Finalmente, las tres frecuencias convergieron a un valor entre 21.53 y 21.61, lo cual sí refleja el comportamiento real del código ejecutado.

Es muy importante destacar que el código intenta quitar el *overhead* inicial del ciclo *for* y simular la cantidad de instrucciones pero se aleja del valor real, por lo que numéricamente el análisis puede ser un poco alejado de la realidad pero permite analizar el comportamiento real del sistema.

2. Realice un profiling del programa y determine cuál es la operación que utiliza más ciclos de reloj. Determine cuál es el *bottleneck* de su programa.

Para determinar la operación que más ciclos de reloj usa, se utilizó el código en la carpeta “codigo_profiling_ej3” disponible en el repositorio y los valores obtenidos también en la pregunta 1.

La lógica implementada fue determinar $X = 1000$ en el código “profiling_ej3.c” para luego ejecutar las operaciones del código 1 1000 veces y medir la cantidad de ciclos individualmente. De esta manera, se podría aproximar qué operación toma más ciclos de reloj de forma particular.

Se ejecutó el código y en base a los resultados se obtuvo el gráfico de la figura 4:

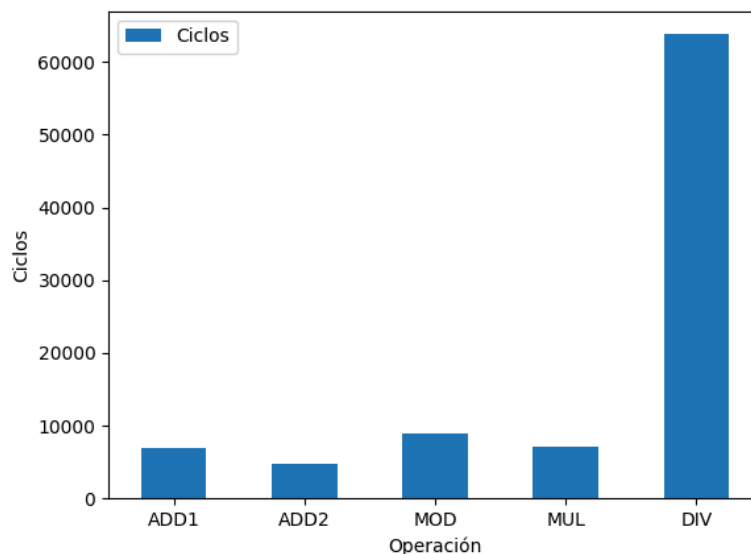


Figura 4: Número de ciclos por operación
Fuente: Elaboración propia

Tal como se muestra en la figura 4, la operación de división es la más costosa en ciclos de reloj.

Esto se debe principalmente a que computacionalmente la división es muy compleja, o mucho más que sumar o multiplicar. Esto porque se usan

métodos iterativos para calcularla. Además, en el código de *profiling* se define explícitamente que debe ser un *float*, lo cuál también favorece a que aumente su consumo de ciclos.

Considerando lo anteriormente mencionado y entendiendo que en el código de la carpeta “implementacion_pseudo_code” se calculan las mismas operaciones (sumas, módulo, multiplicación y división) en cada iteración con misma frecuencia, es posible comparar directamente su impacto en el tiempo total de ejecución y consumo de ciclos.

En consecuencia, se concluye que la división representa el *bottleneck* del programa, ya que es la que consume la mayor cantidad de ciclos de reloj y, por lo tanto, domina el tiempo de ejecución.

3. Utilizando *menuconfig* modifique la frecuencia del MCU y analice el impacto en tiempo, potencia, energía y ciclos de reloj.

Tiempo

Se puede apreciar el impacto de las diversas frecuencias en el tiempo de ejecución en el siguiente gráfico:

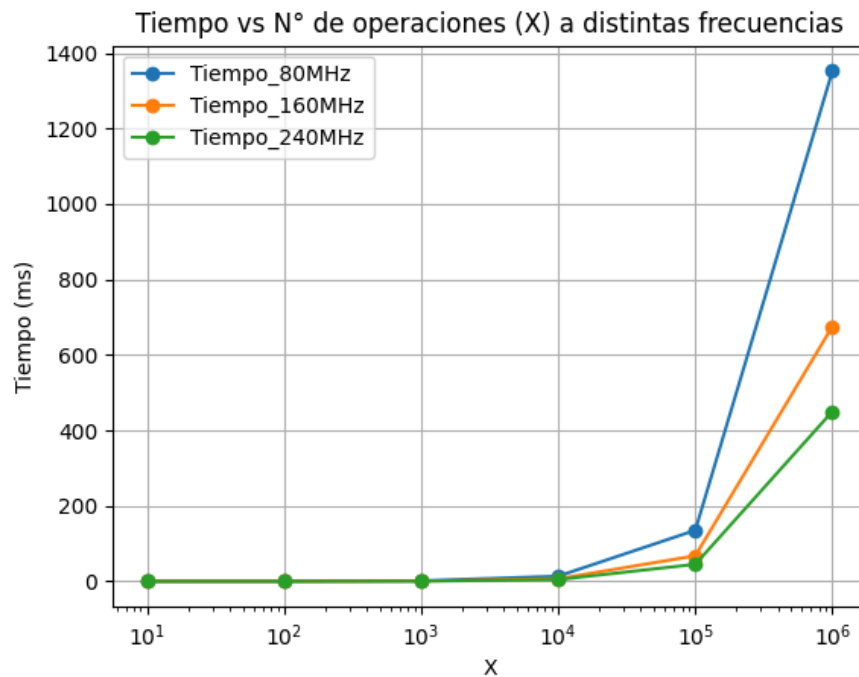


Figura 5: Impacto frecuencia MCU en tiempo de ejecución
Fuente: Elaboración propia

Debido a que la duración de 1 ciclo de reloj es equivalente a $1/\text{frecuencia}$, si se aumenta esta última, se tendrán ciclos más cortos. Esto último significa que las instrucciones se podrán ejecutar más rápido y por ende, terminar el programa en menor tiempo.

En conclusión, aumentar la frecuencia del MCU, disminuye el tiempo de ejecución porque el procesador puede ejecutar más ciclos por segundo.

Potencia

A continuación se presenta un gráfico sobre la variación en la potencia del MCU en base al cambio de sus frecuencias de operación:

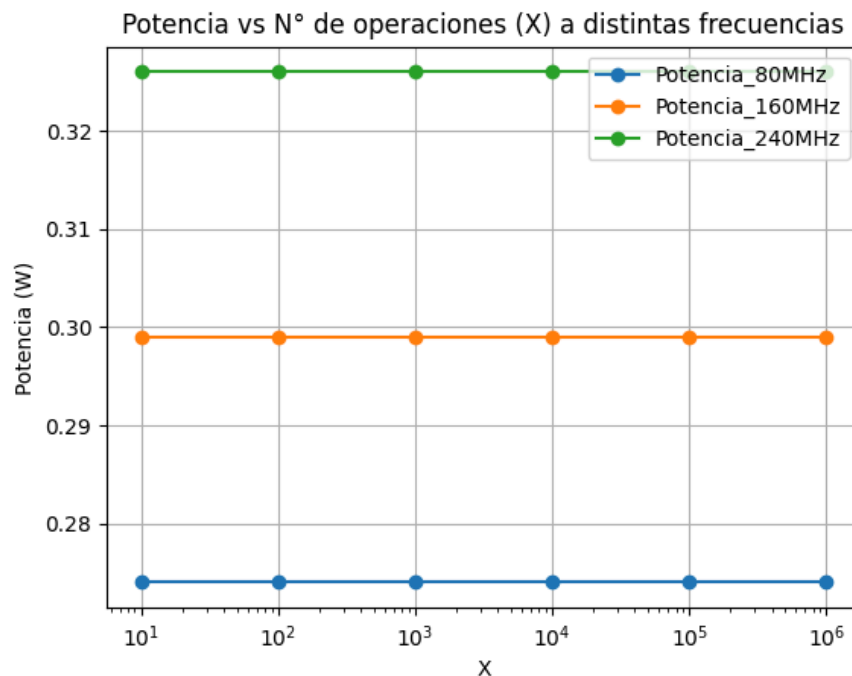


Figura 6: Potencia vs N° de operaciones a distintas frecuencias de MCU
Fuente: Elaboración propia

Tal como se puede observar en la figura número 6, se puede determinar que a mayor frecuencia, mayor potencia consumida. Esto en palabras simples se puede explicar con que la frecuencia aumenta la actividad por segundo del procesador, por lo que implica que va a haber un mayor consumo de energía y por ende una mayor potencia.

Por otro lado, como era de esperar, la potencia se mantiene constante (aproximadamente) al variar el número de operaciones ya que es una tasa de consumo energético. De esta manera, la cantidad de operaciones ejecutadas no afecta directamente la potencia, sino que influye en el tiempo de ejecución y, por lo tanto, en la energía total consumida.

Energía

Mediante los valores de potencia obtenidos en la medición anterior (0.274 W para 80 MHz, 0.299 W para 160 MHz y 0.326 W para 240 MHz), se procedió a calcular la energía del MCU, según la frecuencia que se estuviera utilizando con la fórmula $E = P * t$, considerando $t = 1/60$ para trabajar en Wh (y después dividir en 1000 para mWh).

Se midió directamente en la ESP32-S3 pero la medición no era muy precisa, por lo que se aproximaron y luego se compararían con los valores calculados mediante potencia y fórmula. La medición se realizó en el tiempo de 1 minuto. A continuación se presenta un gráfico con los resultados obtenidos:

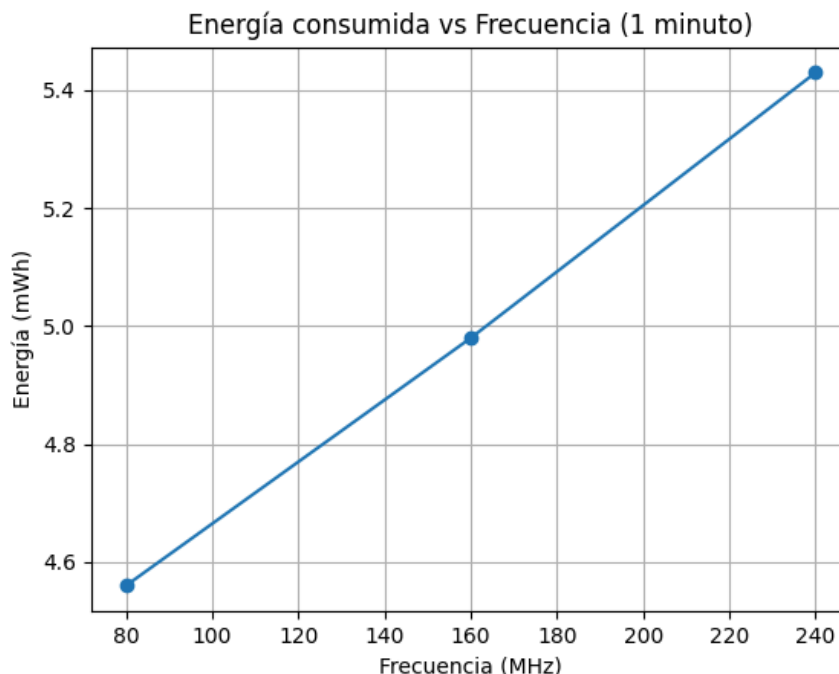


Figura 7: Energía consumida en un minuto vs frecuencias de MCU

Fuente: Elaboración propia

Si se aumenta la frecuencia, la energía consumida también lo hará si esta es medida en un intervalo de tiempo determinado (1 minuto en este caso). Esto se debe a que la potencia aumenta al subir la frecuencia, por lo que, al mantenerse constante el tiempo de medición, un mayor consumo de potencia se traduce directamente en una mayor energía consumida. De esta forma, la energía depende de la potencia del sistema y del tiempo de ejecución, por lo

que en este caso el aumento en la frecuencia significa un mayor consumo energético total.

CPI

A continuación se presenta el gráfico 8, el cual representa la variación del CPI en base a la frecuencia utilizada en el MCU:

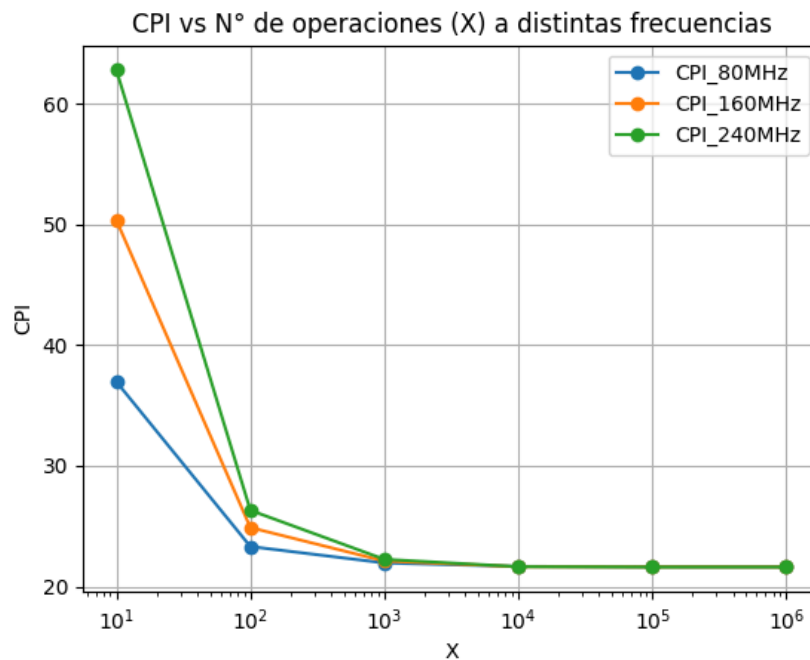


Figura 8: CPI vs N° de operaciones a distintas frecuencias de MCU

Fuente: Elaboración propia

Tal como se había mencionado en la pregunta 1 de este informe, se puede apreciar que una cantidad pequeña de instrucciones hace que el valor del CPI se dispare considerablemente, mientras que el número de operaciones ya pasa el valor de 1000, el valor de CPI se estabiliza y converge a pesar de la frecuencia del MCU.

Se puede establecer que no hay una relación directa entre frecuencia del microcontrolador y los "Clocks Per Instruction" ya que este último se calcula en base a la cantidad de ciclos dividido en cantidad de instrucciones, donde ambos valores son independientes de la frecuencia del MCU. Tal como se pudo ver en el apartado de tiempo de esta pregunta, este parámetro si se ve modificado según el valor de frecuencia.

A pesar de esto, en valores de X (o instrucciones) pequeños, el CPI se dispara. Esto, tal como se explicó antes, es debido a que el *overhead* del programa (inicialización, creación de variables, cachés vacías, etc.) es muy grande en comparación al número de instrucciones totales. Al momento de tener que calcular el CPI matemáticamente, esto hace que el valor crezca rápidamente.